When does additional flash capacity actually improve LLM serving?

Zack Yu et al. arXiv:2609.39131v2, 5 Oct 2026

Yu-Jui Liang
Advisor: Hsiang-Yun Cheng
National Taiwan University | Academia Sinica
27 Oct 2026

HBF helps high-throughput serving mainly by expanding usable device memory, with memory architecture, placement, and scheduling co-designed.

Source: Characterizing High Bandwidth Flash for LLM Serving, paper summary; supplied research brief.

Why Does Agentic Serving Need More Device Memory?

Why is HBM capacity a bottleneck?

Limited HBM capacity

- Model weights
- Active request KV
- Reusable completed-session KV

Completed-session KV can be useful again at the next turn.

One agent session

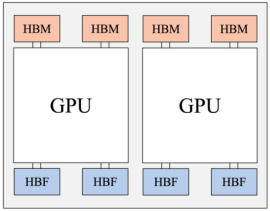
Turn 1
↓
KV generated
↓
Tool / environment wait
↓
Turn 2 reuses Turn-1 prefix

HBF: larger package-local capacity. **Cost:** higher read energy, slow writes, finite endurance.

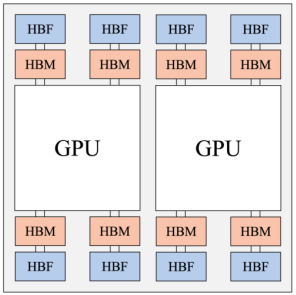
The opportunity is capacity, but HBF cannot be treated as free extra memory.

Source: Characterizing High Bandwidth Flash for LLM Serving, motivation; Table 1.

Two Ways to Add HBF: Shared-Site vs. H^3



(a) Sandisk: 4 HBM + 4 HBF



(b) H^3: 8 HBM + 8 HBF

Shared-site: fixed 8 sites

4 HBM + 4 HBF, for example.
More HBF sacrifices HBM capacity / bandwidth.

H^3: 8 HBM + 8 HBF

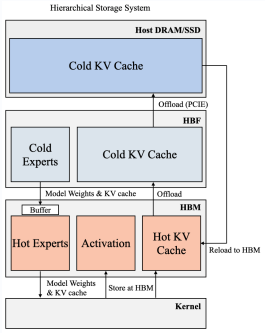
HBF connects behind HBM. Assumed HBM buffering stages HBF data and hides latency.

Full HBM resources, but more stacks and costlier packaging.

Shared-site trades HBM for HBF; H^3 adds HBF without giving up the original HBM capacity.

Source: Characterizing High Bandwidth Flash for LLM Serving, Sec. 4.1; Fig. 2.

How Does the HBM–HBF–Host Hierarchy Work?



HOT: HBM

Frequent weights, activations, hot index, active / hot KV. New KV and reloaded prefixes enter here.

HBF → host DRAM → SSD: COLD

Request-level LRU demotes older KV. Host holds colder state; SSD is coldest.

Placement depends on the object

Activations stay in HBM. Weights stay fixed per configuration. Split MoE: frequent experts in HBM, remaining experts in HBF.

Index-first under HBM pressure

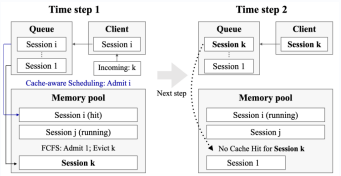
Indexer scans context; attention reads Top-K KV. Demote eligible KV before the hotter index.

The hierarchy treats different objects differently; HBF is not a generic overflow bucket.

Source: Characterizing High Bandwidth Flash for LLM Serving, Fig. 3; placement and cache policy.

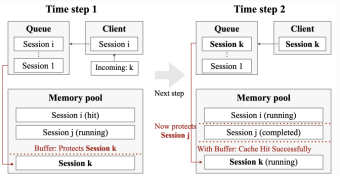
Why Is Cache-Aware Scheduling Alone Not Enough?

Vanilla cache-aware



Cache hit first $\rightarrow$ too much active state
$\rightarrow$ growing locked KV $\rightarrow$ evict reusable KV
$\rightarrow$ future turn loses its cache hit.

Buffered cache-aware



$L + A_i \leq bC, \quad b = 0.9.$
C: usable KV/index capacity after weights;
L: locked active state; A_i: added locked state.

10% is an admission threshold, not a permanently reserved physical partition. Active contexts can keep growing after admission.

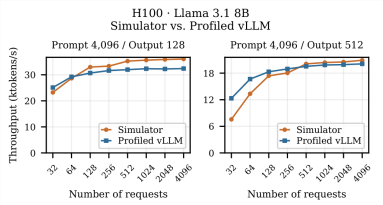
Less admission pressure $\rightarrow$ less churn $\rightarrow$ retained prefixes $\rightarrow$ fewer reloads / repeated prefills / HBF writes.

Sacrificing some instantaneous concurrency can improve total throughput by preserving reuse.

Source: Characterizing High Bandwidth Flash for LLM Serving, Figs. 4–5; buffered admission policy.

What Exactly Does the Simulator Model?

HBM-only validation against profiled vLLM



Llama-3.1-8B on H100; 32–4096 single-turn requests. Mean input 4096; output 128 / 512.

Throughput MAPE: 9.14%.
H100/B200 profiling calibrates fitted kernel latency models. Main experiments use B200 compute.

Request trace
↓
Scheduler / cache state
↓
Kernel latency models
↓
HBM / HBF / DRAM / SSD
↓
PCIe / NVLink transfer model
↓
Discrete request-level execution
↓
Completion time / energy / writes

Request-level serving simulation with calibrated kernel models. Not a cycle-level GPU simulator.

HBF behavior is extrapolated from assumed specifications.

Source: Characterizing High Bandwidth Flash for LLM Serving, Fig. 1; simulator methodology.

Experimental Setup: What Workload Stresses HBF?

Session $\neq$ turn

Session = one agent conversation / trajectory.
Turn = one LLM request within it.

Session A: Turn 1 $\rightarrow$ gap $\rightarrow$ Turn 2 $\rightarrow$ gap $\rightarrow$ Turn 3

LMCache workload

Original: **24,880 turns / 767 sessions**.
25k experiments: 771 sessions.
100k experiments: 3,080 sessions.

Qwen3-32B: scheduling / endurance
Llama-3.1-405B: dense model
GLM-5.2: sparse MoE

Same 3,080 sessions, longer conversations

1×: 100k turns, original length pattern



5×: 500k turns, five concatenated copies



Copies continue the preceding context.
Mean input: **22,558** $\rightarrow$ **89,125 tokens**.

Long-context stress, not merely more independent requests.

The 5× trace tests HBF as per-session KV state grows much larger.

Source: Characterizing High Bandwidth Flash for LLM Serving, workload construction; Table 3; bars are schematic.

Q1: Can Scheduling Make HBF KV Writes Sustainable?

Qwen3-32B: H^3 with 8 HBM + 8 HBF

Scheduling	Completion (s)	KV writes (TB)	Device hit	Life (yr)
FCFS	13,552.76	106.79	32.84%	1.21
Cache-aware	10,930.78	21.81	86.87%	4.77
Cache-aware + 10%	**10,419.17**	**6.69**	**96.46%**	**14.82**

Adding the buffer

HBF writes: 21.81 → 6.69 TB

-69.33%

Completion time: -4.68%

Estimated lifetime, not measured lifetime

FCFS → buffered: 1.21 → 14.82 years.

Assumes 100k P/E cycles, uniform wear, and unit write amplification.

Preserving headroom reduces both churn and write pressure. Endurance depends strongly on serving policy.

Source: Characterizing High Bandwidth Flash for LLM Serving, Table 4; simulator lifetime assumptions.

Q2: Does More HBF Capacity Help Dense LLM Serving?

Llama-3.1-405B: 8 GPUs, TP8, 1× / 100k turns, 10% buffer

Configuration	Time (s)	Energy (MJ)	TPOT (ms)
HBM-only, HBM weights	39,666	53.60	90.85
H^3, HBM weights	21,439	40.70	438.61
H^3, HBF weights	21,423	42.06	438.27

Extra capacity helps throughput

H^3 / HBM weights vs. HBM-only:
45.95% less completion time;
about **24.1% less modeled energy**.

TPOT: **90.85** → **438.61 ms**.
Higher throughput ≠ lower request latency.

Moving hot weights adds little

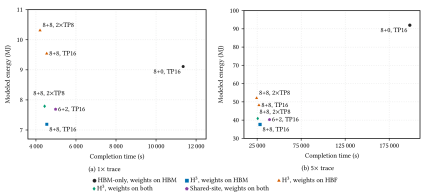
21,439 vs. 21,423 s: almost equal.
40.70 vs. 42.06 MJ: HBF weights cost **3.33% more energy**.

Dense weights stay hot; colder KV can reside in HBF.

Dense Llama benefits mainly from extra KV capacity, not from moving hot weights to HBF.

Source: Characterizing High Bandwidth Flash for LLM Serving, Table 5; modeled energy excludes static / background power.

Q3: How Do Placement and Parallelism Change MoE Results?



GLM-5.2: HBM-only → H^3, TP16 / HBM weights

1×: 11,354 / 9.11 → 4,543 / 7.19 (s / MJ)
59.99% less time.
5×: 198,411 / 91.99 → 28,279 / 37.62
7.0× faster; 59.10% less energy.

Shared-site: 6+2 best at 1×. At 5×, 4+4 takes 33,459 s vs. 39,019 s for 6+2 (**14.25% faster**); 6+2 uses **4.01% less energy**.

DPA attention → TP/EP MoE

DPA: local requests/KV; combine rank batches for MoE.
TP shards tensor computation; EP routes tokens to selected expert GPUs.
TP16/EP16 + DPA: one 16-GPU replica.
2×TP8/EP8 + DPA: two 8-GPU replicas.

Fastest ≠ lowest energy

5×, H^3 2×TP8 / HBF weights:
24,385 s, but **52.06 MJ** vs. 37.62 MJ.

Longer contexts favor HBF capacity; placement and parallelism depend on the objective.

Source: Characterizing High Bandwidth Flash for LLM Serving, Fig. 6; Table 6; full configurations in Backup PDF 18.

Where Do the Gains Come From, and What Did the Paper Prove?

More device capacity $\rightarrow$ larger batches
$\rightarrow$ weight reuse $\uparrow$ $\rightarrow$ weight-read energy $\downarrow$

More KV retained $\rightarrow$ prefix reuse $\uparrow$
$\rightarrow$ reload / repeated prefill $\downarrow$

HBF itself is not lower-energy memory.
Read: HBM 3.4 vs. HBF 6.8 pJ/bit.

Energy saving (pp)	Llama 1×	GLM 5×
Weight reads	+15.22	+55.14
KV/index reads	−13.16	−0.61
Avoided prefill	+20.25	0
Other	+1.74	+4.58
Net (%)	**24.06**	**59.10**

H^3 8+8; component rounding may differ from net.

1. Capacity matters

Expand the on-device working set for long-context throughput.

2. Policy matters

Placement and buffered admission control churn and write wear.

3. Co-design matters

Co-design HBF with the scheduler rather than treat it as passive overflow.

Simulation-based results, not production HBF hardware measurements.

Source: Characterizing High Bandwidth Flash for LLM Serving, Table 7; Table 1; reported net values retained.

HBM vs. HBF Assumptions

Modeled assumptions / extrapolations

Parameter	HBM	HBF
Capacity / stack	24 GB	375 GB
Read bandwidth	1000 GB/s	1000 GB/s
Write bandwidth	1000 GB/s	8 GB/s
Read startup	0.1 μs	20 μs
Write startup	0.1 μs	250 μs
Read energy	3.4 pJ/bit	6.8 pJ/bit
Write energy	3.4 pJ/bit	100 pJ/bit
P/E cycles	—	100,000

HBF adds capacity with substantial write, latency, and energy costs.

Source: Characterizing High Bandwidth Flash for LLM Serving, Table 1.

Shared-Site vs. H^3 Service-Time Model

Shared-site: 8 total stack sites

$$T_{\text{shared}} = \max\left(\frac{D_H}{mB}, \frac{D_F}{(8-m)B}\right)$$

H^3: conservative serialized model, 8 HBM + 8 HBF

$$T_{H^3} = \frac{D_H}{8B} + \frac{D_F}{8B}$$

D_H, D_F: bytes served from HBM / HBF.

B: modeled per-stack bandwidth; m: shared-site HBM stack count.

H^3 benefits from retaining 8 HBM stacks and adding 8 HBF stacks. Serialization itself is not an advantage over parallel access.

Source: Characterizing High Bandwidth Flash for LLM Serving, Sec. 4.1; service-time model.

Energy Model

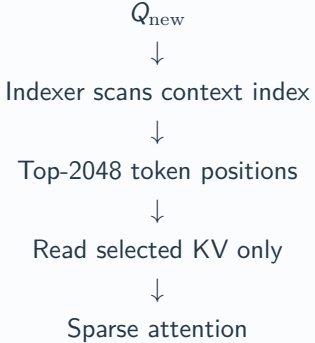
$$E = \underbrace{\sum_j N_j e_j}_{\text{arithmetic}} + \underbrace{\sum_m 8(R_m e_m^r + W_m e_m^w)}_{\text{memory reads / writes}} + \underbrace{E_{\text{comm}}}_{\text{communication}}$$

- N_j: operation count; e_j: energy per arithmetic operation.
- R_m, W_m: bytes read / written in tier m; factor 8 converts bytes to bits.
- e_m^r, e_m^w: per-bit read / write energy.
- E_{comm}: modeled communication energy.

Static / background power is not included. Reported MJ values are not complete wall-plug server energy.

Source: Characterizing High Bandwidth Flash for LLM Serving, energy model.

Sparse Attention and Index-First Placement



Different access intensities

The indexer sees the full context.

Attention reads a selected KV subset.

The index is therefore hotter than sparse KV.

Under HBM pressure, demote eligible KV before index.

Top-2048 KV selection is not MoE Top-k expert routing.

Source: Characterizing High Bandwidth Flash for LLM Serving, sparse-attention and index-first placement descriptions.

TP / EP / DPA: A Four-GPU Example

DPA: attention home ranks

GPU0	Request A + KV_A
GPU1	Request B + KV_B
GPU2	Request C + KV_C
GPU3	Request D + KV_D

TP: shard one tensor computation

The same large tensor computation runs in shards across multiple GPUs.

EP: illustrative expert ownership

GPU0: E0, E1 GPU1: E2, E3
GPU2: E4, E5 GPU3: E6, E7

Request A finishes attention on GPU0.
Router selects **E1 + E5**:

E1: local on GPU0.
E5: send token activation to GPU2.
Return / combine expert outputs before the next attention layer.

Request/KV has an attention home rank; token activations temporarily move to the GPUs holding selected experts.

Source: Characterizing High Bandwidth Flash for LLM Serving, parallelism description; illustrative 4-GPU example, not an evaluated configuration.

Does the Paper Dynamically Swap Experts?

No. Weight placement is fixed per evaluated configuration.

Offline expert-routing profile $\rightarrow$ prioritize frequent experts for HBM $\rightarrow$ remaining experts in HBF. Runtime routing may select an HBF-resident expert and incur its weight-read cost. It does not imply moving the whole expert to HBM.

Our possible extension, not a contribution of this paper

- Per-layer expert migration
- Dynamic expert caching / runtime expert replacement
- Expert prefetch prediction

Distinguish dynamic token routing from fixed expert-weight placement.

Source: Characterizing High Bandwidth Flash for LLM Serving, weight-placement assumptions.

Full GLM Table 6

GLM-5.2: time (s) / modeled energy (MJ) / TPOT (ms)

Configuration / weights	1× trace Time	1× trace Energy	1× trace TPOT	5× trace Time	5× trace Energy	5× trace TPOT
HBM-only 8+0 TP16 / HBM	11,354	9.11	218.72	198,411	91.99	191.93
H^3 8+8 TP16 / HBM	4,543	7.19	279.15	28,279	37.62	355.63
H^3 8+8 TP16 / HBF	4,538	9.54	278.01	27,036	48.17	333.66
H^3 8+8 2×TP8 / HBF	4,204	10.31	230.67	24,385	52.06	280.43
H^3 8+8 2×TP8 / Split	4,433	7.79	252.61	25,661	40.89	307.66
Shared 2+6 TP16 / Split	6,073	8.51	342.12	35,809	44.81	406.36
Shared 4+4 TP16 / Split	5,264	7.99	322.72	33,459	41.97	423.71
Shared 6+2 TP16 / Split	4,981	7.69	302.87	39,019	40.28	410.47

$X + Y$: HBM + HBF stacks per GPU. TP16: EP16 + DPA. Each TP8 replica: EP8 + DPA.

Split: hot experts fixed in HBM, others in HBF.

The fastest configuration and the lowest-energy configuration differ.

Source: Characterizing High Bandwidth Flash for LLM Serving, Table 6.

Scope and Limitations

Serving scope

- High-throughput workload completion
- Latency-constrained serving outside scope
- Autoregressive decoding only
- No speculative decoding

Calibration and extrapolation

Kernel models calibrated on H100/B200. HBF and large-batch results involve extrapolation.

Hardware / energy scope

- No production HBF controller
- No complete package-power model
- Wear leveling not fully modeled
- Not every activation / staging write modeled

Treat speedups and lifetime as architecture / systems characterization under explicit assumptions, not production HBF measurements.

Source: Characterizing High Bandwidth Flash for LLM Serving, scope and modeling limitations.